

Proyecto 2 - Historia Clinica

Magaly Rosales

Pregunta 1: entidades relacionadas y CRUD

Entrega integrada | 9 de octubre de 2026

Aporte original conservado

El commit 2355ca3656d7b463bfa98ce90efa6979fc47ef8a pertenece a Magaly Rosales Porras. Agrega la relacion bidireccional HistoriaClinica-Atencion y evita la recursion al serializar JSON mediante JsonManagedReference y JsonBackReference.

HistoriaClinica es el lado inverso: OneToMany(mappedBy="historiaClinica", cascade=ALL, orphanRemoval=true). Atencion es el lado propietario:ManyToOne y clave foranea historia_clinica_id. Una historia agrupa varias atenciones; cada atencion pertenece a una historia.

La capa Controller recibe HTTP, Service aplica reglas y transacciones, Repository usa Spring Data JPA. Hibernate traduce el modelo a las tablas historia_clinica y atencion. El proyecto tambien conserva Paciente-HistoriaClinica como relacion OneToOne.

JsonManagedReference permite incluir las atenciones dentro de la historia. JsonBackReference evita incluir de nuevo toda la historia dentro de cada atencion. Se previene el ciclo historia-atencion-historia sin eliminar la relacion de base de datos.

Correcciones de integracion y demostracion

Se inicializo la coleccion de atenciones con ArrayList para que una historia nueva pueda recibir registros sin una coleccion nula. Se mantuvieron las anotaciones JSON y el commit original.

Se completo la edicion y eliminacion de atenciones verificando que el registro pertenezca a la historia indicada. Un ID de otra historia devuelve 404. Al eliminar se actualiza la coleccion del padre para respetar orphanRemoval.

El puerto MySQL dejo de depender de un 3307 fijo: DB_PORT permite 3306 o 3307. Esta correccion permite compartir el mismo codigo con instalaciones diferentes sin modificar application.properties.

Flujo para exponer: ingresar como admin o medico, registrar un paciente ficticio, crear su historia, abrir Ver historial de atenciones, registrar una atencion, editarla y verificar su persistencia. Para eliminar, usar solamente un registro de demostracion.

En Postman demostrar consulta JSON de la historia, modificacion de una atencion y rechazo de una operacion que use otra historia. En SQL consultar historia_clinica y atencion usando el ID creado. No usar datos personales reales en capturas.

Guion y preguntas del profesor

Guion: Mi aporte original conecta HistoriaClinica y Atencion en ambos sentidos. El lado propietario guarda la clave foranea y el lado inverso mantiene la coleccion. Las referencias JSON evitan recursion. Durante la integracion se inicializo la coleccion, se completo el CRUD relacionado y se flexibilizo el puerto de base de datos.

¿Que hace mappedBy? Identifica el atributo historiaClinica de Atencion que es propietario de la relacion; evita una segunda relacion o tabla intermedia innecesaria.

¿Que hace orphanRemoval? Una atencion retirada de la coleccion persistida puede eliminarse como registro dependiente. Cascade ALL propaga operaciones del padre. Debe usarse con cuidado porque eliminar el padre puede eliminar hijos.

¿Por que comprobar la historia al editar? Para impedir que un endpoint de una historia modifique una atencion de otra. El ID existente no basta: se valida tambien la pertenencia.

Archivos a mostrar: src/main/java/com/tecsup/historiaclinica/model/HistoriaClinica.java, model/Atencion.java, service/HistoriaClinicaService.java y el controlador de historias. Buscar las clases por nombre en el IDE.

Ejecucion y evidencia

Clonar <https://github.com/JeryBn/Evaluacion1Coello> y abrir el pom.xml de la raiz. Requisitos: Java 21, Maven y MySQL/MariaDB. No iniciar la carpeta antecedentes como otra aplicacion.

Crear la base historia_clinica e importar database/esquema-proyecto2.sql solamente en una base vacia. El esquema tiene 13 tablas y no publica registros ni claves. En una base existente, respaldar antes de actualizar.

Seguir el README: definir el perfil demo y las tres variables DEMO_ADMINISTRADOR_PASSWORD, DEMO_MEDICO_PASSWORD y DEMO_RECEPCIONISTA_PASSWORD. Iniciar Spring Boot e ingresar con admin, medico o recepcion. Las claves son definidas por quien instala y no se reemplazan si los usuarios ya existen.

Importar postman/Proyecto-2.postman_collection.json y configurar baseUrl y las claves de los tres usuarios. Ejecutar en orden. La coleccion obtiene CSRF antes de las operaciones de escritura y crea datos ficticios.

Resultado comprobado: 13 pruebas automatizadas exitosas. La coleccion ejecutada mediante Newman produjo 47 solicitudes HTTP y 30 aserciones sin fallos. Newman ejecuta el formato Postman; no debe confundirse con una ejecucion manual de su interfaz.

El reparto organiza la exposicion, no cambia la autoria de los commits. La implementacion restante se completo con asistencia de Codex en la copia de Jery. La guia utiliza indicaciones individuales: confirmar con el docente el formato de entrega del grupo.

Evidencia real: historia demo

Captura del entorno local integrado. Datos ficticios de demostracion; sin claves publicadas.

Inicio

Pacientes

Auditoria

Usuarios

Roles

Historias clínicas

Atención médica

← Atrás

Adelante →

admin

Cerrar sesion

Detalle de Historia Clínica

N° Historia:

HC-000005

Fecha de apertura:

2026-10-08

Estado:

ACTIVA

Datos del paciente

Nombres:

Demostracion

Apellidos:

Proyecto Dos

DNI:

DEMO-1791521183113

Sexo:

F

Teléfono:

000000000

Ver historial de atenciones

Ver antecedentes y alergias

Continuar a

atención médica

Volver al listado

Evidencia real: atenciones

Captura del entorno local integrado. Datos ficticios de demostracion; sin claves publicadas.

Inicio

Pacientes

Auditoria

Usuarios

Roles

Historias clínicas

Atención médica

← Atrás

Adelante →

admin

Cerrar sesion

Historial de Atenciones

Registrar nueva atención

Motivo de la atención

Consulta general

Observaciones (opcional)

Registrar Atención

Atenciones registradas

Fecha	Motivo	Observaciones	Acciones
2026-10-08	Control actualizado		▶ Editar Eliminar

Volver al listado

Continuar a atención médica